

Postgres e SQL - Parte #02:

Consultando múltiplas tabelas com JOINS

Banco de Dados

Prof. Dr. Carlos Melo

 carlos.melo@upe.br



Acompanhamento do Projeto (15 min)

- Status atual do projeto
- Dificuldades encontradas
- Próximos passos



O Problema

- Precisamos combinar dados de múltiplas tabelas em uma única consulta
- O Gerente precisa de um relatório que mostre o Nome do Empregado e o Nome do Departamento onde ele trabalha.
 - O nome do funcionário está na tabela empregado
 - O nome do departamento está na tabela departamento



```
1 SELECT nome FROM empregado;  
2 SELECT nome FROM departamento;
```





O Problema

- Até agora, nossos **SELECT**s focaram em apenas uma origem por vez, mas como juntar isso em uma única resposta?



O que é um **JOIN**?

- O **JOIN** é a operação que permite combinar linhas de duas ou mais tabelas baseando-se em uma relação entre colunas (geralmente PK e FK):
 - empregado.numero_departamento → departamento.numero
- A ideia por trás dos **JOINS** é a teoria de conjuntos

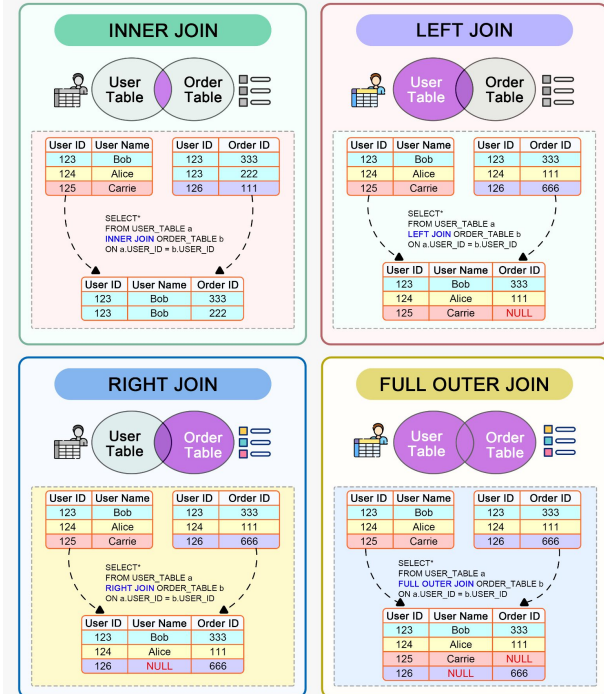


Os principais tipos de JOINS

- **INNER JOIN** -> Apenas correspondências
 - **LEFT JOIN** -> Tudo à esquerda
 - **RIGHT JOIN** -> Tudo à direita
 - **FULL OUTER JOIN** -> Tudo de ambos
- > A tabela à **esquerda** é a que aparece primeiro no **FROM**

Top 4 Types SQL Joins

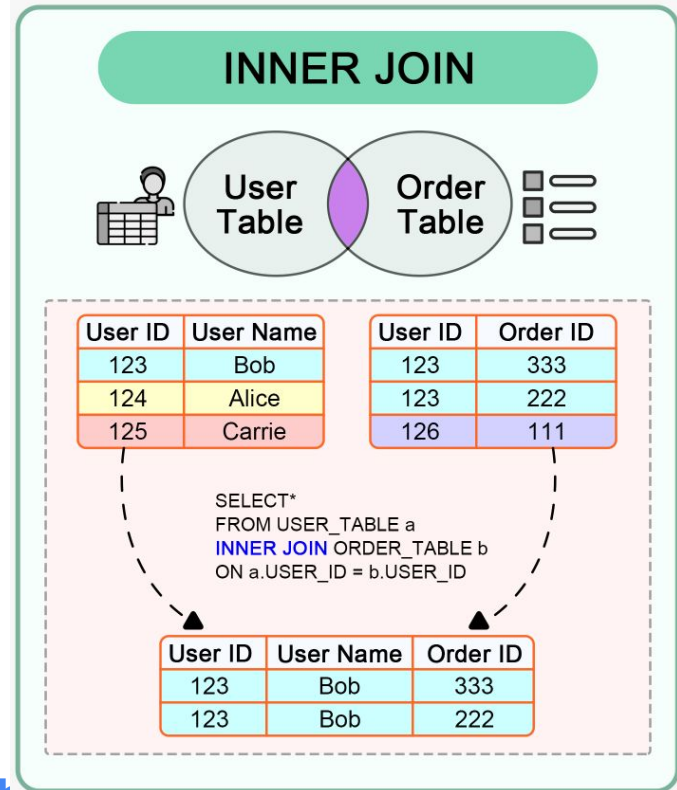
ByteByteGo



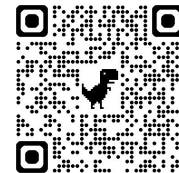


INNER JOIN: Intersecção

- É o tipo mais comum de junção
- Retorna apenas as linhas que possuem correspondência em ambas as tabelas, ou seja:
- SÓ APARECEM REGISTROS QUE EXISTEM NAS DUAS TABELAS
- empregado sem departamento → NÃO aparece
- departamento sem empregado → NÃO aparece



INNER JOIN: Sintaxe



```
1 SELECT tabela1.coluna, tabela2.coluna
2 FROM tabela1
3 INNER JOIN tabela2 ON tabela1.fk = tabela2.pk;
```



```
1 SELECT empregado.nome, departamento.nome
2 -- colunas que queremos
3
4 FROM empregado
5 -- tabela principal
6
7 INNER JOIN departamento
8 -- tabela que será combinada
9
10 ON empregado.numero_departamento = departamento.numero
11 -- condição de ligação
```




Cláusula **ON**

- Enquanto o **WHERE** realiza filtros gerais (ex: preço > 100)
- O **ON** é uma condição específica para associar linhas de duas tabelas distintas, é ele quem ensina ao banco como aquelas duas tabelas estão conectadas.



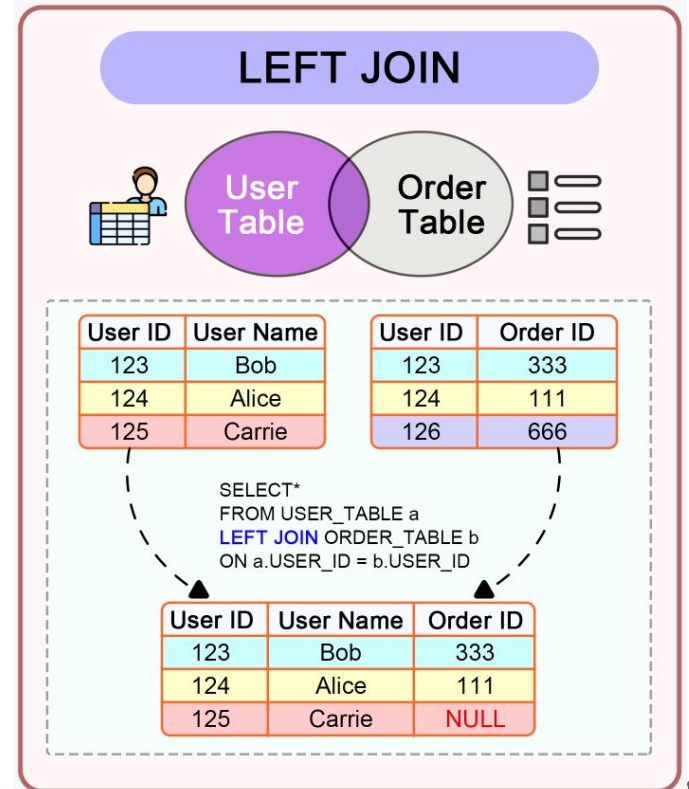
INNER JOIN: Intersecção

- Se cadastrarmos um novo departamento (ex: Inovação), que ainda não tem nenhum funcionário associado, esse departamento não aparecerá no relatório gerado pelo **INNER JOIN**
- Isso é bom ou ruim?



LEFT JOIN: Lado esquerdo

- Retorna todos os registros da tabela principal (à esquerda do comando **JOIN**) + suas correspondências na tabela da direita:
 - departamento (esquerda) → sempre aparece
 - empregado (direita) → pode virar NULL





LEFT JOIN: Lado esquerdo

- Se não houver correspondência, o resultado do lado secundário será preenchido com **NULL**, ou seja, não existe correspondência
- Exemplo:
 - Garantir que todos os Departamentos apareçam no relatório, mesmo os que não tem funcionários
- **INNER JOIN** → oculta dados sem correspondência
- **LEFT JOIN** → preserva dados da esquerda



Listando todos os departamentos



```
1 SELECT d.nome AS setor, e.nome AS funcionario
2 -- d = departamento (sempre aparece)
3
4 FROM departamento d
5 LEFT JOIN empregado e
6 -- e pode ser NULL
7
8 ON d.numero = e.numero_departamento;
```

- Qual coluna pode apresentar correspondências nulas?



Queries Complexas

- Imagine que o RH precisa de um relatório completo unindo 4 tabelas: empregado, departamento, projeto e empregado_projeto
- Consultas com muitos **JOINS** ficam difíceis de ler, manter e repetir
- E se pudéssemos salvar essa consulta e tratá-la como se fosse uma tabela comum?

View



- Uma **View** é uma consulta (**SELECT**) armazenada que pode ser usada como uma "tabela virtual"
- Diferente das tabelas base, uma **View** geralmente não duplica os dados em disco; apenas salva o SQL de como buscar aqueles dados
 - Tabela → guarda dados
 - **View** → guarda uma consulta
- É excelente para a segurança, uma vez que podemos dar acesso à **View** para o estagiário do setor sem dar detalhes pessoais e individuais de cada colaborador



Criando uma **View**



```
1 CREATE VIEW v_relatorio_projetos AS
2 -- criando uma view
3
4 SELECT e.nome AS funcionario, p.nome AS projeto, ep.horas
5 -- dados que queremos expor
6
7 FROM empregado e
8 INNER JOIN empregado_projeto ep ON e.cpf = ep.cpf_empregado
9 INNER JOIN projeto p ON ep.codigo_projeto = p.codigo;
```




Criando uma **View**

- Depois de criada, usamos a **View** como se fosse uma tabela

```
1 SELECT
2     e.nome AS funcionario,
3     p.nome AS projeto,
4     ep.horas
5 FROM empregado e
6 INNER JOIN empregado_projeto ep
7     ON e.cpf = ep.cpf_empregado
8 INNER JOIN projeto p
9     ON ep.codigo_projeto = p.codigo
10 WHERE ep.horas > 20;
```

```
1 SELECT * FROM v_relatorio_projetos WHERE horas > 20;
```



Exercícios

Insira o código do novo "Departamento de Inovação" para o teste do LEFT JOIN dar certo:

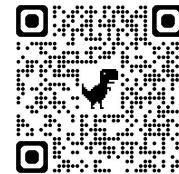
```
INSERT INTO departamento (numero, nome) VALUES (4,  
'Inovação');
```

1. Escreva uma consulta que retorne o nome de todos os departamentos lado a lado com os nomes dos empregados que trabalham neles (Use INNER JOIN).
2. O Diretor quer ver a lista de TODOS os projetos da empresa e, se houver, os funcionários alocados neles. O relatório não pode esconder projetos vazios (Dica: Use LEFT JOIN).



Exercícios

1. O RH detestou ter que digitar aquele monte de JOINS. Transforme a consulta da Atividade 1 em uma **View** chamada v_equipe_setor.
2. Faça um **SELECT** na sua nova **View**, filtrando apenas as pessoas que trabalham na 'Tecnologia da Informação'



Prof. Dr. Carlos Melo
<https://calendly.com/prof-casm>

 carlos.melo@upe.br